1.10 Scopes and Namespaces

When computing a sum with the syntax x + y in Python, the names x and y must have been previously associated with objects that serve as values; a NameError will be raised if no such definitions are found. The process of determining the value associated with an identifier is known as ***name resolution***.

Whenever an identifier is assigned to a value, that definition is made with a specific ***scope***. Top-level assignments are typically made in what is known as ***global*** scope. Assignments made within the body of a function typically have scope that is ***local*** to that function call. Therefore, an assignment, x = 5, within a function has no effect on the identifier, x, in the broader scope.

Each distinct scope in Python is represented using an abstraction known as a ***namespace***. A namespace manages all identifiers that are currently defined in a given scope. Figure 1.8 portrays two namespaces, one being that of a caller to our count function from Section 1.5, and the other being the local namespace during the execution of that function.

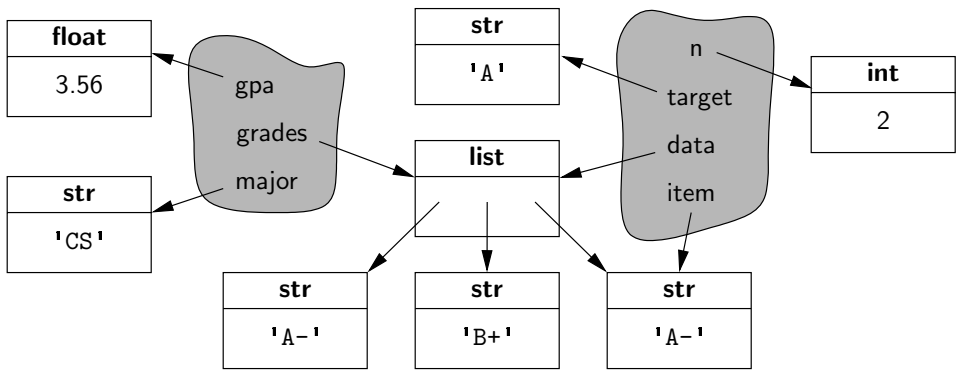


Figure 1.8: A portrayal of the two namespaces associated with a user's call count(grades, 'A'), as defined in Section 1.5. The left namespace is the caller's and the right namespace represents the local scope of the function.

Python implements a namespace with its own dictionary that maps each identifying string (e.g., 'n') to its associated value. Python provides several ways to examine a given namespace. The function, dir, reports the names of the identifiers in a given namespace (i.e., the keys of the dictionary), while the function, vars, returns the full dictionary. By default, calls to dir( ) and vars( ) report on the most locally enclosing namespace in which they are executed.

When an identifier is indicated in a command, Python searches a series of namespaces in the process of name resolution. First, the most locally enclosing scope is searched for a given name. If not found there, the next outer scope is searched, and so on. We will continue our examination of namespaces, in Section 2.5, when discussing Python's treatment of object-orientation. We will see that each object has its own namespace to store its attributes, and that classes each have a namespace as well.

First-Class Objects

In the terminology of programming languages, ***first-class objects*** are instances of a type that can be assigned to an identifier, passed as a parameter, or returned by a function. All of the data types we introduced in Section 1.2.3, such as int and list, are clearly first-class types in Python. In Python, functions and classes are also treated as first-class objects. For example, we could write the following:

```
scream = print       # assign name 'scream' to the function denoted as 'print'
scream('Hello')     # call that function
```

In this case, we have not created a new function, we have simply defined scream as an alias for the existing print function. While there is little motivation for precisely this example, it demonstrates the mechanism that is used by Python to allow one function to be passed as a parameter to another. On page 28, we noted that the built-in function, max, accepts an optional keyword parameter to specify a non-default order when computing a maximum. For example, a caller can use the syntax, max(a, b, key=abs), to determine which value has the larger absolute value. Within the body of that function, the formal parameter, key, is an identifier that will be assigned to the actual parameter, abs.

In terms of namespaces, an assignment such as scream = print, introduces the identifier, scream, into the current namespace, with its value being the object that represents the built-in function, print. The same mechanism is applied when a user-defined function is declared. For example, our count function from Section 1.5 beings with the following syntax:

```
def count(data, target):
  ...
```

Such a declaration introduces the identifier, count, into the current namespace, with the value being a function instance representing its implementation. In similar fashion, the name of a newly defined class is associated with a representation of that class as its value. (Class definitions will be introduced in the next chapter.)